



System design

E-commerce Marketplace (e.g. Amazon)

★ Premium



Medium



40 mins



1.27k done

Report an issue 🛠️

Asked at these companies



Amazon



Shopify



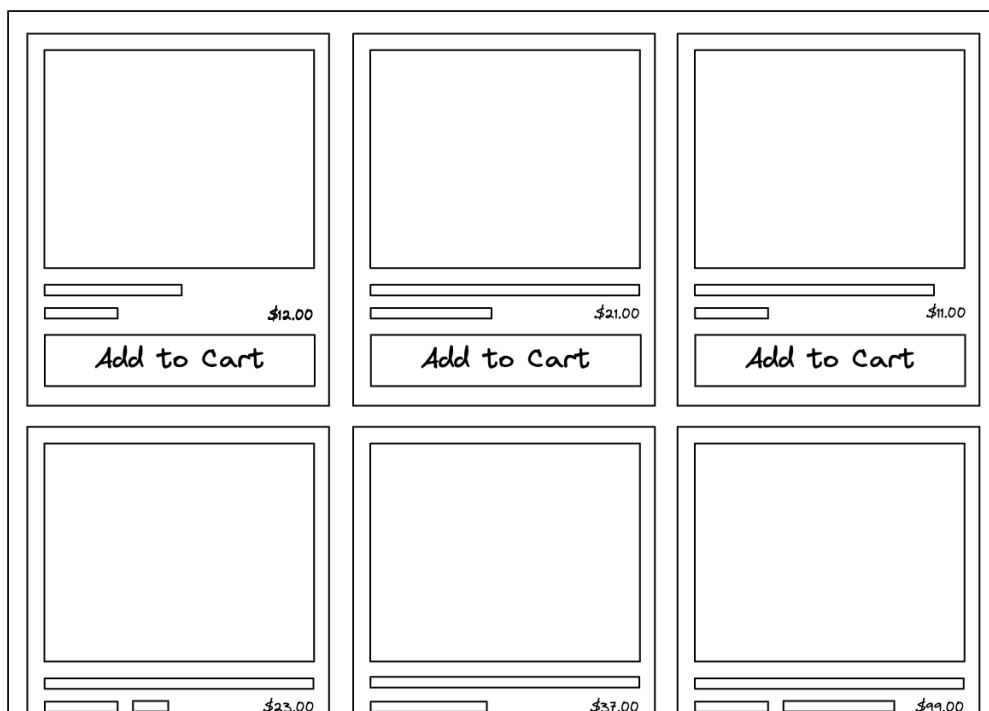
Stripe



PayPal

Question

Design an e-commerce website that allows users to browse products and purchase them.



\$12.00

Add to Cart

5 items in the cart

Qty: 1 | Delete

\$12.00

Qty: 3 | Delete

\$69.00

Qty: 1 | Delete

\$33.00

Total: \$114.00

Checkout

Address

John Doe

Country

Street

City

State

ZIP

Card Information

1234 1234 1234 1234

MM/YY

CVC

Order

Real-life examples

- <https://www.amazon.com>
- <https://www.ebay.com>
- <https://www.walmart.com>
- <https://www.flipkart.com>

Requirements exploration

Before designing the storefront, we scope the exercise by clarifying which user journeys are in play, which devices to target, and which commerce concerns (payments, localization, accessibility) fall inside the interview.

What are the core features to be supported?

- Browsing products.
- Adding products to cart.
- Checking out successfully.

What are the pages in the website?

- Product listing page (PLP)
- Product details page (PDP)
- Cart page
- Checkout page

What product details will be shown on the PLPs and PDPs?

- PLPs: Product name, Product image, Price.
- PDPs: Product name, Product images (multiple), Product description, Price.

What do the user demographics look like?

International users of a wide age range: US, Asia, Europe, etc.

What are the non-functional requirements?

For key routes, target p75 LCP ≤ 2.5 seconds, INP ≤ 200 ms, and CLS ≤ 0.1 , measured separately for mobile and desktop in important markets. Cart and checkout mutations should also have route-specific latency and error-rate targets.

What devices will the application be used on?

All possible devices: laptop, tablets, mobile, etc.

Do users have to be signed in to purchase?

Users can purchase as a guest without being signed in.

Exercise Beta

A storefront design focuses on the customer journey from discovery through purchase. Which boundaries keep that work focused? Select all that apply.

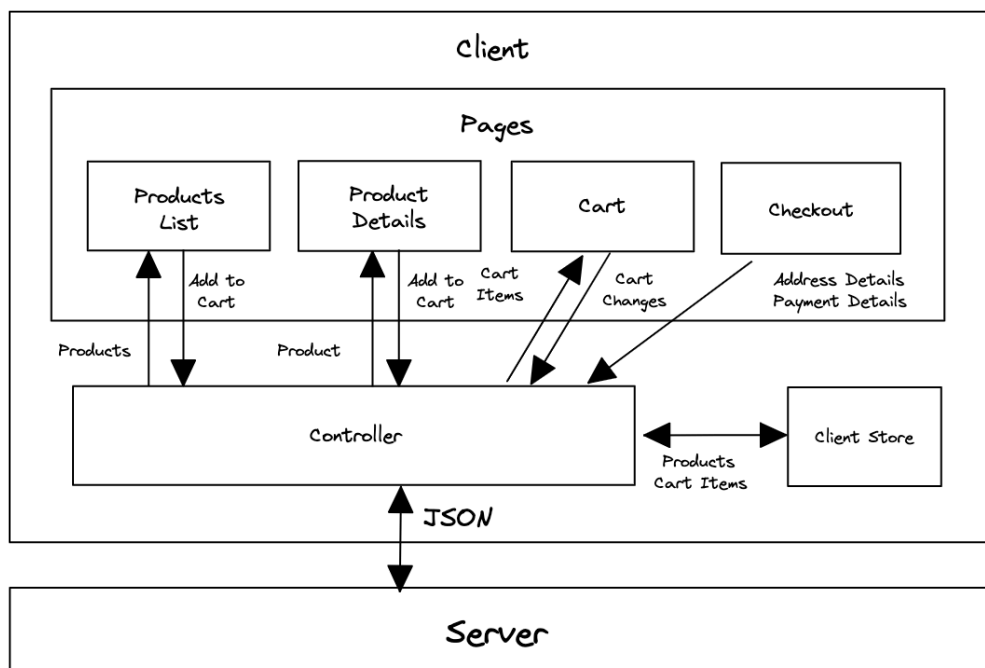
- 1 Support international shoppers across desktop and mobile, including checkout without mandatory account creation. ☐
- 2 Make seller catalog administration, fulfillment queues, and payout reporting part of the first-version storefront design. ☐
- 3 Cover product listing and detail pages, a shared cart, checkout, and the successful order result. ☐
- 4 Treat personalized recommendation ranking and sponsored-product auctions as required parts of the baseline listing page. ☐
- 5 Define separate loading, responsiveness, layout-stability, and mutation-reliability targets for public catalog and checkout routes. ☐



Reveal answer

Check answer ↩

Architecture / high-level design



The diagram connects the product, cart, and checkout pages to a controller and shared client store backed by server APIs. Its "Payment Details" label refers only to an

opaque payment-method ID created by provider-hosted fields; raw card data never enters the controller or store.

Component responsibilities

- **Server:** Provides HTTP APIs to fetch product data and cart items, modify the cart, and create orders.
- **Controller:** Controls the flow of data within the application and makes network requests to the server.
- **Client Store:** Stores data needed across the whole application. Since there are many pages with some amount of overlapping data, a client store is useful for sharing data across sections of a page and across pages.
- **Pages:**
 - **Product List:** Displays a list of product items that can be added to the cart.
 - **Product Details:** Displays details of a single product along with additional details.
 - **Cart:** Displays added cart items and allows changing of quantity and deleting added items.
 - **Checkout:** Displays address and payment forms the user has to complete in order to place the order.

Server-side rendering or client-side rendering?

Firstly, let's understand what the two terms mean:

- **Server-side rendering (SSR):** The traditional way of building websites where the server fetches all necessary data, uses it to create the final markup, and sends down the HTML needed every time a user visits a page. Most of the rendering work is done on the server.
- **Client-side rendering (CSR):** The server sends down initial HTML which contains the JavaScript to bootstrap the application. The client then fetches the necessary data, combines it with templates, and creates the final page, all within the browser. CSR is typically used with Single-page Application models, and subsequent navigations don't require a full page refresh. Most of the rendering work is done on the client.

Possible benefits of SSR for public catalog routes:

- Product content can appear in the initial HTML without waiting for client JavaScript, although actual performance still depends on server latency, caching, and hydration cost.
- Reserving layout space in the initial HTML can reduce layout shift.
- SSR allows for personalization of pages (user-specific content) as opposed to static site generation. Personalization is an important factor for conversion as e-commerce platforms scale up.

Downsides of SSR:

- SSR can add server work and time to first byte, although navigation after hydration can still remain client-side.

SEO is important for public PLPs and PDPs, so SSR or pre-rendering is a strong default for those routes rather than a guarantee for every page.

✓ **Prefer SSR or pre-rendering for public PLPs and PDPs**

Unlike a personalized feed, public catalog pages benefit from crawlable product content in the initial HTML. SSR and pre-rendering are both reasonable choices; select between them based on freshness, personalization, caching, and server cost.

Single-page application (SPA) or Multi-page application (MPA)?

SPA and MPA describe navigation, while CSR and SSR describe rendering, so the choices are not inherently coupled. Universal rendering (or SSR with hydration) renders the initial HTML on the server, after which rendering and navigation can become client-side.

Most universally rendered sites use popular UI frameworks (e.g. React, Vue, and Angular), and the page will need to be hydrated (add event handlers) after initial load. Hydration also brings about the double data problem.

Which application architecture should be used? SPA and MPA are both viable; choose rendering per route based on crawlability, first-load performance, freshness, and interaction needs.

Real-world technologies to implement the following rendering and architecture choices:

	SSR	CSR
SPA	Next.js, Remix, Nuxt	Vite-based SPA
MPA	Ruby on Rails, Django*	Less common, but possible

* Refers to the default mode of server-side web frameworks, which render HTML, and routing is done on the server side.

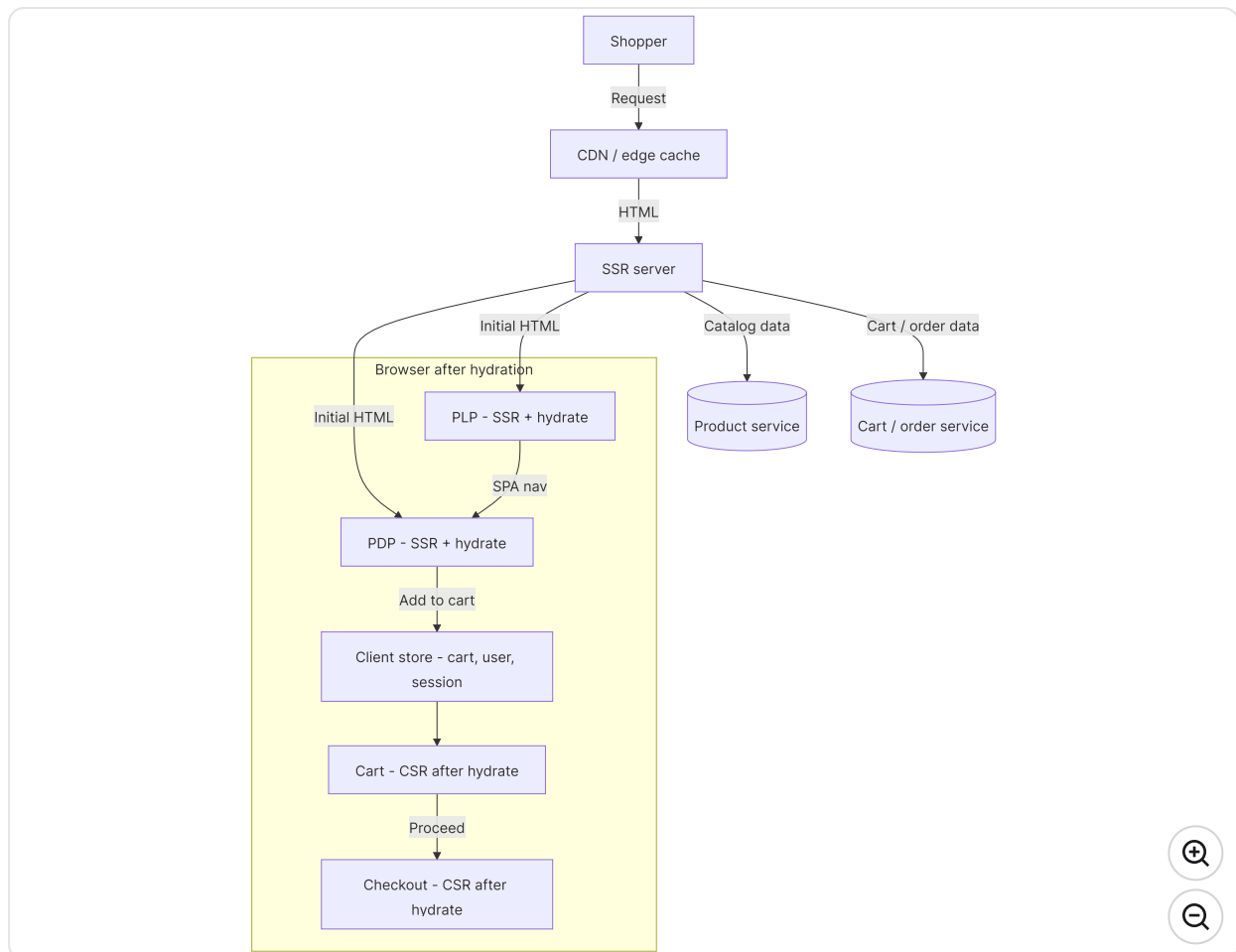
What top e-commerce sites use

Let's take a look at e-commerce sites in the wild and their rendering choices:

	Architecture	Rendering	UI Framework
Amazon	MPA	SSR	In-house
eBay	MPA	SSR	<u>Marko</u>
Walmart	SPA	SSR	React
Flipkart	SPA	SSR	React

All these e-commerce sites use SSR! This suggests the importance of using SSR for e-commerce sites.

The discussion below assumes that we'll be using a universal rendering approach of SSR + SPA. Concretely, this means the catalog surfaces (PLP, PDP) prioritize SSR for SEO and first paint, while interactive transactional surfaces (Cart, Checkout) lean more on the client after hydration.



Page-type rendering strategy across the storefront

Exercise Beta

Which rendering architecture best supports crawlable public product pages while preserving an instant shared cart across client navigation?

- 1 Statically generate every catalog and checkout route at build time, refreshing the deployment whenever product data changes.
- 2 Server-render every catalog, cart, and checkout transition and discard the hydrated client store on each navigation.
- 3 Use client-side rendering for every route, but emit product metadata from the server for crawlers.
- 4 Server-render public listing and product pages, then hydrate interactive features while keeping cart and account flows client-rich.



Reveal answer

Check answer ↩

Data model

There are quite a number of entities involved in an e-commerce website due to the complexity of the user flow spanning multiple pages.

Entity	Source	Belongs To	Fields
ProductList	Server	Product Listing Page	products (list of Product s), pagination (pagination metadata)
Product	Server	Product Listing Page, Product Details Page	id , name , description , unit_price , currency , primary_image , image_urls
Cart	Server	Client Store	items (list of CartItem s), total_price , currency
CartItem	Server	Client Store	quantity , product , price (line total), currency
AddressDetails	User input (Client)	Checkout Page	name , country , street , city , etc.
PaymentMethod	Payment provider	Checkout Page	Opaque provider method/session ID plus non-sensitive display metadata such as brand and last four digits
Order	Server	Checkout success state	id , items (list of OrderItem s), total_price , currency , status ,

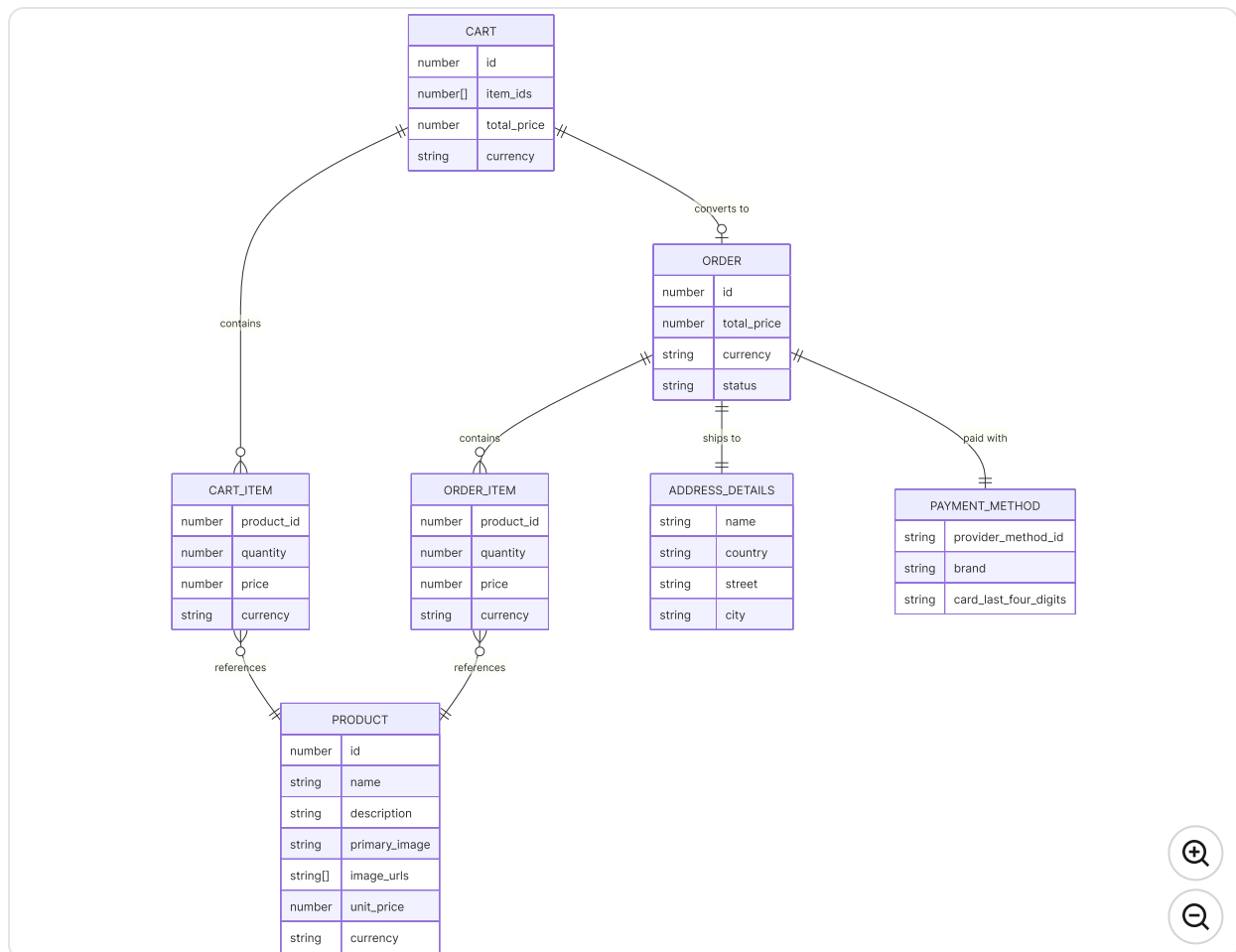
Entity	Source	Belongs To	Fields
			address_details , payment_method
OrderItem	Server	Order	product_id , quantity , price (line total), currency

Few points for discussion:

- The `Cart` entity belongs to the Client Store because some websites might want to show the number of cart items in the navbar or have a popup that allows users to quickly access the cart items and make modifications. If there's no such need, then it's acceptable for the cart to belong to the Cart page.
- `Cart` and `CartItem` have `total_price` and `price` fields respectively fetched as part of the server response, instead of having the client compute the price (multiply the quantity by the `unit_price`), to give the flexibility of applying discounts due to bulk purchases or use of promo codes. The price computation logic is defined on the server, so the final price should be computed on the server, and the client should rely on the server to calculate the total price instead of making its own calculations.
- Since our website has an international audience, we should have localized prices in the user's currency, hence the `currency` field.

The examples use decimal prices for readability; a production API should document an exact money representation such as `{ amountMinor, currency }` or a decimal string.

The diagram below summarizes how these entities relate once the shopper moves from browsing a product to placing an order. Both `CartItem` and `OrderItem` reference `Product`, while `AddressDetails` and a tokenized `PaymentMethod` are attached to the order record.



Core e-commerce entities and their relationships

For simplicity, each product maps to one cart line. If the interviewer expands into marketplace behavior, discuss variants, seller offers, fulfillment choices, and immutable order-line snapshots.

★ Treat the server as the source of truth for cart totals and prices

Client-side multiplication of quantity times unit_price silently breaks once bulk discounts, promo codes, tax, or regional pricing enter the picture. Always render the price and total_price returned by the server and reconcile from the cart returned by each mutation.

Exercise Beta

Which ownership choices match the storefront data model? Select all that apply.

1

Keep the current cart in shared client state when the navbar, drawer, and cart page all consume it.

☐

2

Create the order entity when the browser reaches the confirmation route, before the order service responds.

☐

3

Represent each cart line with product identity, quantity, currency, and a server-returned line price.

☐

4

Keep editable address input separate from the server-created order returned after checkout succeeds.

☐

5

Persist the formatted price shown on the product card and use it to calculate the eventual cart total.

☐

Reveal answer

Check answer ↩

Interface definition (API)

We need the following HTTP APIs:

1. Product Information

- Fetch products listing
- Fetch a particular product's detail

2. Cart Modification

- Fetch the current cart
- Add a product to the cart
- Change quantity of a product in the cart
- Remove a product from the cart

3. Complete the order

We will omit discussion of the APIs between the client components because the data format and functionalities are similar to the HTTP APIs.

The server owns the cart identity. `GET /cart` resolves a signed-in shopper from the account and a guest from an opaque, high-entropy token stored in a `Secure`, `HttpOnly`, appropriately `SameSite` cookie. Cart and order APIs operate on that resolved server cart rather than trusting a browser-supplied cart ID.

Fetch products listing

Field	Value
HTTP Method	GET
Path	/products
Description	Fetches a list of products.

Parameters

Parameter	Type	Description
size	number	Number of results per page
page	number	Page number to fetch
country	string	Market or shipping-country hint; the server returns the canonical price and currency

Sample response

```
1  {
2    "pagination": {
3      "size": 5,
4      "page": 2,
5      "total_pages": 4,
6      "total": 20,
7    },
8    "products": [
9      {
10       "id": 123, // Product ID.
11       "name": "Cotton T-shirt",
12       "primary_image": "https://www.greatcdn.com/img/t-shirt.jpg",
13       "unit_price": 12,
14       "currency": "USD",
```

```
15     },
16     // ... More products.
17   ],
18 }
```

We use offset-based pagination here as opposed to cursor-based pagination because:

1. Having page numbers is useful for navigating between search results and jumping to specific pages.
2. Use a stable ordering because ranking, price, or availability changes can otherwise shift products between pages.
3. It's useful to know how many total results there are.

For a more in-depth comparison between offset-based pagination and cursor-based pagination, refer to the [News Feed system design article](#).

Fetch product details

Field	Value
HTTP Method	GET
Path	/products/{productId}
Description	Fetches the details of a product.

Parameters

Parameter	Type	Description
productId	number	ID of product to be fetched
country	string	Market or shipping-country hint; the server returns the canonical price and currency

Sample response

```
1  {
2    "id": 123, // Product ID.
3    "name": "Cotton T-shirt",
4    "description": "A lightweight cotton T-shirt.",
5    "primary_image": "https://www.greatcdn.com/img/t-shirt.jpg",
6    "image_urls": [
7      "https://www.greatcdn.com/img/t-shirt.jpg",
8      "https://www.greatcdn.com/img/t-shirt-black.jpg",
9      "https://www.greatcdn.com/img/t-shirt-red.jpg",
10   ],
11   "unit_price": 12,
12   "currency": "USD",
13 }
```

Add a product to cart

Field	Value
HTTP Method	POST
Path	/cart/products
Description	Add a product to the cart

Parameters

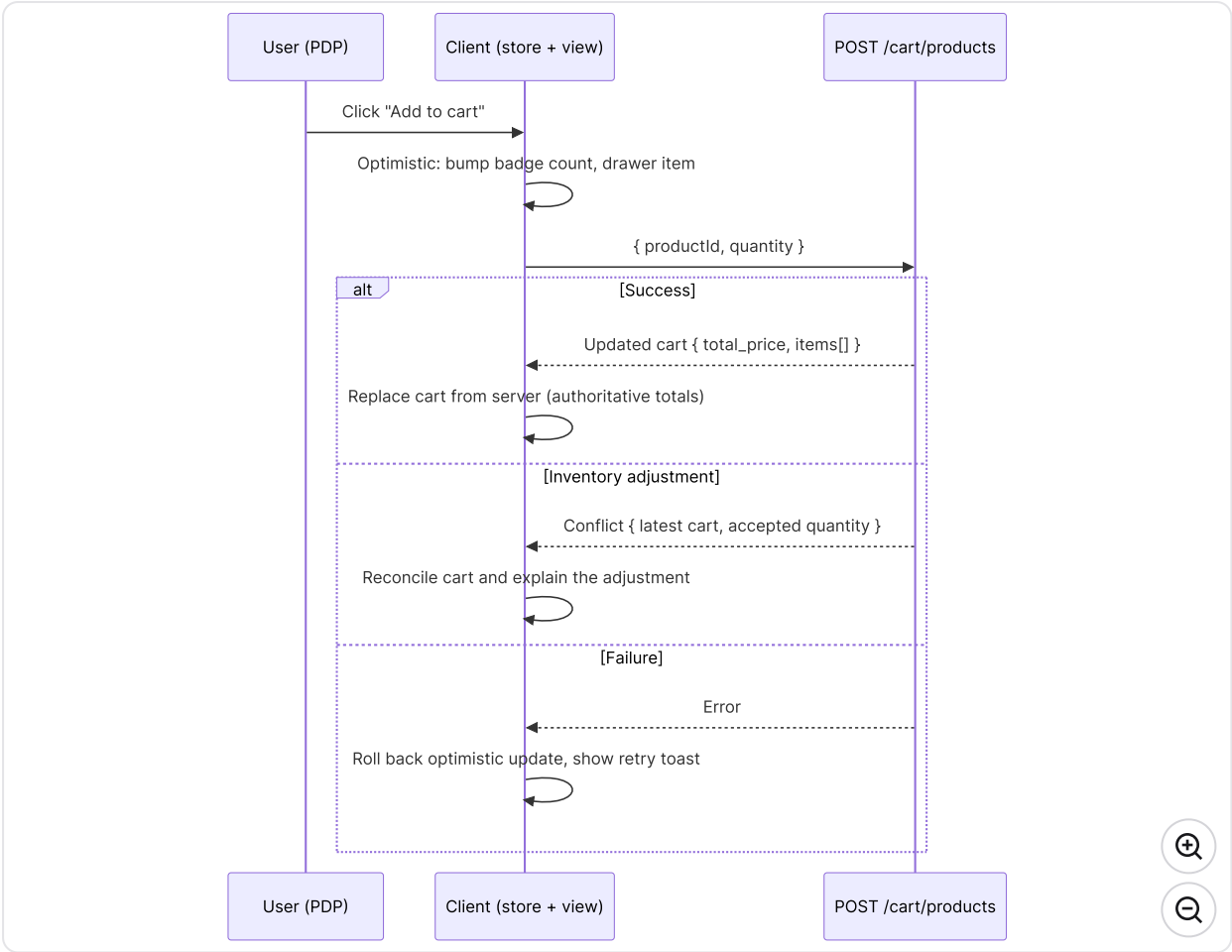
Parameter	Type	Description
productId	number	ID of product to be added
quantity	number	Number of items to be added

Sample response

The updated cart object is returned.

```
1  {
2    "id": 789, // Cart ID.
3    "total_price": 24,
4    "currency": "USD",
5    "items": [
6      {
7        "quantity": 2,
8        "price": 24,
9        "currency": "USD",
10       "product": {
11         "id": 123, // Product ID.
12         "name": "Cotton T-shirt",
13         "primary_image": "https://www.greatcdn.com/img/t-shirt.jpg",
14       },
15     },
16   ],
17 }
```

On the client, add-to-cart benefits from an optimistic update so the cart badge and drawer feel instant, but the server still owns the authoritative totals once discounts and tax kick in. The flow below shows the client applying a temporary update, hitting `POST /cart/products`, and then reconciling against the canonical cart returned by the server.



Add-to-cart flow with optimistic update and server reconciliation

Change quantity of product in cart

Field	Value
HTTP Method	PUT
Path	/cart/products/{productId}
Description	Change quantity of a product in the cart

Parameters

Parameter	Type	Description
productId	number	ID of product to be modified

Parameter	Type	Description
quantity	number	New quantity of the product

Sample response

The updated cart object is returned.

```
1  {
2    "id": 789, // Cart ID.
3    "total_price": 36,
4    "currency": "USD",
5    "items": [
6      {
7        "quantity": 3,
8        "price": 36,
9        "currency": "USD",
10       "product": {
11         "id": 123, // Product ID.
12         "name": "Cotton T-shirt",
13         "primary_image": "https://www.greatcdn.com/img/t-shirt.jpg",
14       },
15     },
16   ],
17 }
```

Remove product from cart

Field	Value
HTTP Method	DELETE
Path	/cart/products/{productId}
Description	Remove a product from the cart

Parameters

Parameter	Type	Description
<code>productId</code>	number	ID of product to be removed

Sample response

The updated cart object is returned.

```
1  {
2    "id": 789, // Cart ID.
3    "total_price": 0,
4    "currency": "USD",
5    "items": [],
6  }
```

Place order

Field	Value
HTTP Method	<code>POST</code>
Path	<code>/order</code>
Description	Create an order from a cart

Parameters

Parameter	Type	Description
<code>address_details</code>	object	Object containing address fields
<code>payment_method_id</code>	string	Opaque identifier created by provider-hosted payment fields

Sample response

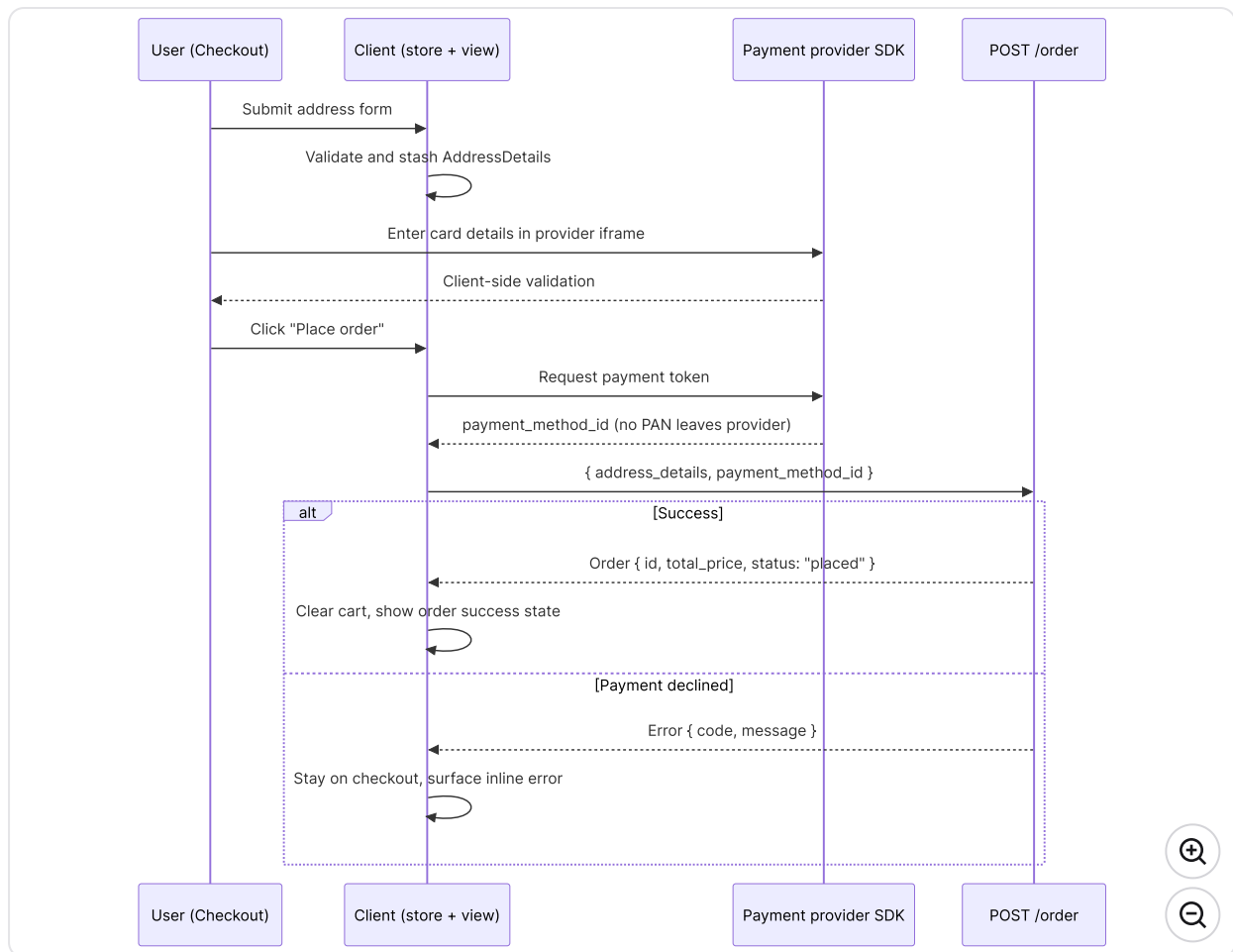
The order object is returned upon successful order creation.

```
1  {
2    "id": 456, // Order ID.
3    "total_price": 36,
4    "currency": "USD",
5    "items": [
6      // ... Same items as per the cart.
7    ],
8    "address_details": {
9      "name": "John Doe",
10     "country": "US",
11     "address": "1600 Market Street",
12     "city": "San Francisco",
13     // ... Other address fields.
14   },
15   "status": "placed",
16   "payment_method": {
17     "brand": "visa",
18     "card_last_four_digits": "1234",
19   },
20 }
```

! Never let raw card numbers touch your servers or client storage

Accepting full PANs through your own `POST /order` endpoint drags the application into PCI DSS scope and makes an XSS or log leak catastrophic. Collect card data in provider-hosted fields and send only the resulting opaque payment method ID; the application API, state, analytics, and logs should never receive a PAN or CVC. This reduces but does not eliminate the merchant's PCI and payment-page security responsibilities.

Concretely, the checkout page hosts the address form directly but embeds the payment fields inside the provider's iframe-backed SDK. The sequence below captures how the client collects an address, exchanges card fields for a payment token with the provider, and then sends only that token to `POST /order`.



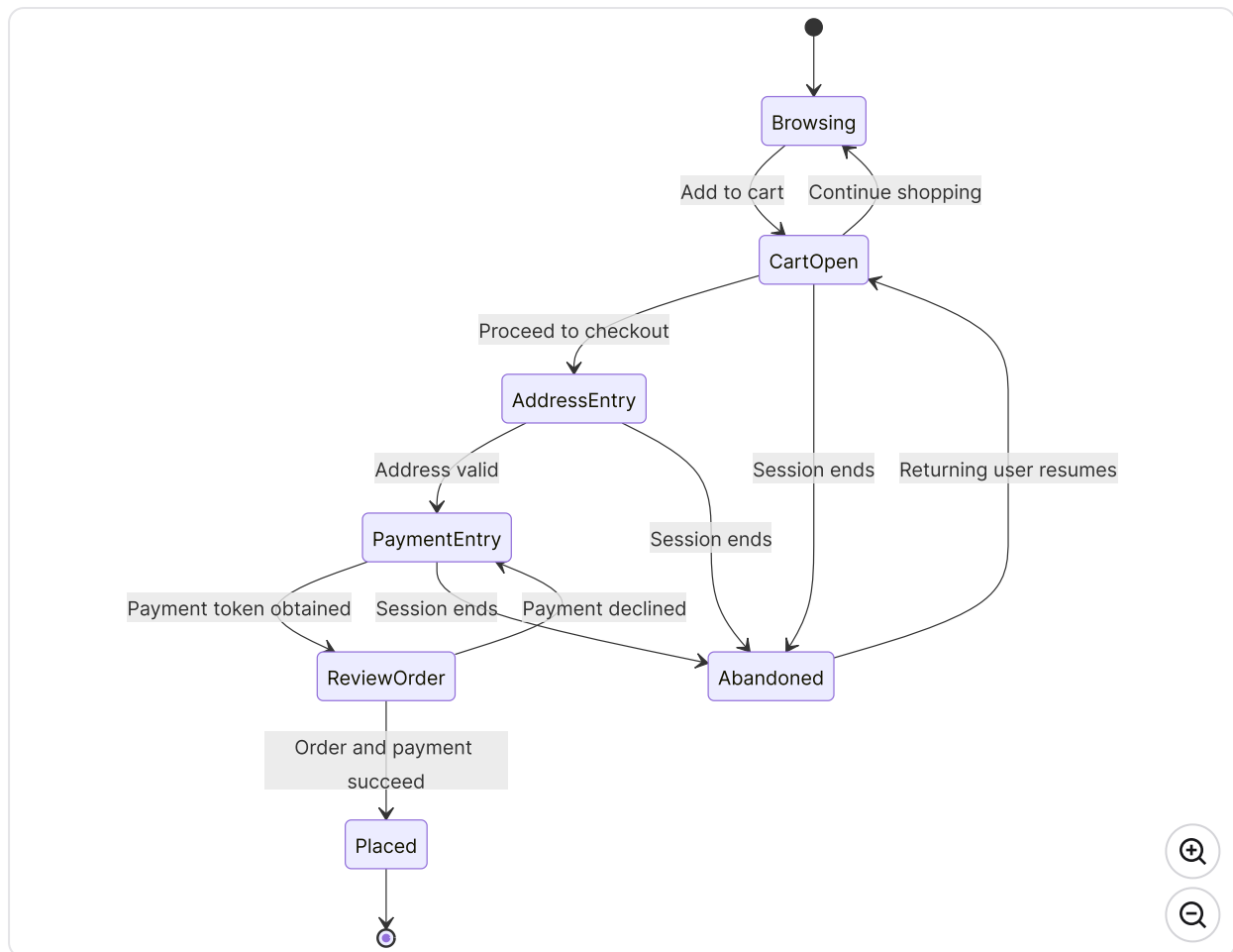
Checkout and place-order flow with tokenized payment

The order endpoint recalculates price and inventory on the server. This answer assumes sequential cart updates and an immediate payment result. If the interviewer asks about reliability, discuss idempotency keys, cart versions, overlapping mutations, asynchronous payment states, authenticated provider webhooks, and compensation.

Notes

- To help returning users resume checkout, save safe address progress. If the interviewer asks about saved payment methods, discuss explicit consent and a customer-bound provider setup; never persist raw card fields in the cart.

The state machine below names the checkout steps explicitly so the UI and the persisted cart can share the same vocabulary. Each transition is driven by a user action or a server response, and any terminal state (placed, abandoned) is a useful analytics boundary for funnel instrumentation.



Checkout and order lifecycle states

Exercise 1 of 5 Beta

The same cart APIs serve signed-in shoppers and guests. Which identity choices prevent one browser from selecting another shopper's cart? Select all that apply.

- 1 Resolve a guest cart from a high-entropy opaque token stored in a `Secure`, `HttpOnly`, appropriately `SameSite` cookie. ☐
- 2 Accept a cart ID from application state and authorize the request when the ID has the expected format. ☐
- 3 Put the guest-cart token in every cart URL so the browser can restore it after navigation. ☐
- 4 Resolve a signed-in shopper's cart from the authenticated account on each request. ☐



Reveal answer

Check answer ←

Optimizations and deep dive

With the core flows in place, this section dives into production concerns that directly affect discovery, conversion, checkout reliability, and customer trust:

- **Performance**: This section explains why page speed matters for commerce and how to prioritize loading work across the shopping flow.
- **Search engine optimization**: This section covers crawlable product pages, metadata, structured data, and search discovery for product listings.
- **Images**: This section discusses responsive product images, modern formats, lazy loading, CDN delivery, and layout stability.
- **Form optimizations**: This section covers checkout form ergonomics, validation, autofill, payment flow, and reducing user friction.
- **Internationalization (i18n)**: This section discusses translations, currencies, addresses, dates, and locale-specific commerce expectations.
- **Accessibility**: This section covers semantic HTML, forms, buttons, images, and keyboard support across shopping and checkout pages.
- **Security**: This section explains client-facing security concerns around authentication, payment data, transport, and third-party scripts.
- **User experience**: This section covers checkout focus, cart behavior, error recovery, and other details that reduce abandonment.

Performance

Performance is absolutely critical for e-commerce websites. Seemingly small performance improvements can lead to significant revenue and conversion increases. A **study by Google and Deloitte** showed that even a 0.1-second improvement in load times can improve conversion rates across the purchase funnel. **web.dev by Google has a long list of case studies of how improving site performance led to improved conversions.**

General performance tips

- Code split JavaScript by routes/pages.

- Split content into separate sections and prioritize above-the-fold content while lazy loading below-the-fold content.
- Defer loading of non-critical JavaScript (e.g. code needed to show modals, dialogs, etc.).
- Prefetch JavaScript and data needed for the next page upon hovering over or focusing links/buttons.
 - Prefetch product details when users hover over or focus items in PLPs.
 - Prefetch checkout page while on the cart page.
- Optimize images with lazy loading and adaptive loading.
- Consider prefetching top search results after measuring whether the transferred data is usually used.

For a deeper caching discussion, separate public catalog responses from private cart and checkout data, then define cache keys, freshness, and invalidation by market and currency.

Core Web Vitals

Know the various Core Web Vital metrics, what they are, and how to improve them.

Measure these targets with real-user monitoring by route, device class, market, and release; use synthetic checks only as regression guards.

- **Largest Contentful Paint (LCP)**: The render time of the largest image or text block visible within the viewport, relative to when the page first started loading.
 - Optimize performance – loading of JavaScript, CSS, images, fonts, etc.
- **Interaction to Next Paint (INP)**: Measures overall interaction responsiveness using the longest observed interaction latency while ignoring defined outliers. INP replaced First Input Delay (FID) as a Core Web Vital on March 12, 2024; FID is no longer measured by Chrome's web-vitals reporting.
 - Reduce the amount of JavaScript executed on page load and break up long tasks so input handlers stay responsive throughout the session, not just on the first interaction.
- **Cumulative Layout Shift (CLS)**: Measures visual stability because it helps quantify how often users experience unexpected layout shifts. A low CLS helps ensure that the page is delightful.

- Include size attributes on images and video elements or reserve space for these elements using CSS `aspect-ratio` to reserve the required space for images while the images are loading. Use CSS `min-height` to minimize layout shifts while elements are lazy loaded.

Exercise Beta

A product page has a strong server response time but poor LCP and INP on mid-range phones. Which changes protect the core buying journey? Select all that apply.

- ☐ 1 Track LCP, INP, and CLS by route, device, and release.
- ☐ 2 Load route-critical catalog and cart code first and defer reviews, recommendations, and secondary widgets.
- ☐ 3 Prefetch full product records for every item linked from the current category so later navigation is instant.
- ☐ 4 Hydrate recommendations, reviews, experimentation, and marketing widgets before enabling the product controls so the page becomes interactive all at once.
- ☐ 5 Cache reusable public responses with explicit freshness and invalidation rules.



Reveal answer

Check answer ↩

Search engine optimization

SEO is extremely important for e-commerce websites as organic search is the primary way people discover products.

- PDPs should have useful `<title>` and description metadata plus open graph tags. Do not rely on the ignored `keywords` meta tag.
- Generate a `sitemap.xml` to tell crawlers the available pages of the website.
- Use JSON structured data to help search engines understand the kind of content on your page. For the e-commerce case, the Product type would be the most relevant.

- Use real links for category and product navigation, stable canonical URLs, and meaningful success or not-found status codes.
- Use semantic markup for elements on the page, which also helps accessibility.
- Keep pages fast for users; good Core Web Vitals can contribute to page experience but do not guarantee rankings.
- Prefer SSR or pre-rendering for durable public catalog content so it remains fast and crawlable without depending on client JavaScript.
- Pre-generate only curated, durable pages for popular searches or lists rather than indexing every filter combination.

✓ **Emit accurate Product JSON-LD on eligible PDPs**

Valid `schema.org/Product` data can make a page eligible for product search features but does not guarantee their display or ranking. Keep `price`, `priceCurrency`, and `availability` synchronized with the visible, current product data.

The [Travel Booking \(Airbnb\) system design article](#) goes into more details about SEO.

Exercise Beta

Which choices help search engines understand product and listing pages? Select all that apply.

- 1

Return meaningful product names, descriptions, prices, and links in the initial HTML.

☐
- 2

Load primary product descriptions and offer data only after the shopper expands each section.

☐
- 3

Use real links for crawlable category and product navigation.

☐
- 4

Provide unique titles, descriptions, canonical URLs, and appropriate structured data.

☐
- 5

Give every sort, filter, and pagination permutation a self-referencing canonical URL.

☐



Reveal answer

Check answer ↩

Images

Images are one of the largest contributors to page size, and serving optimized images is absolutely essential on e-commerce websites, which are image-heavy, with every product having at least one image.

- Serve appropriately compressed modern formats such as AVIF or WebP, with a compatible fallback where required.
- Use `srcset` and `sizes` so the browser selects an image appropriate for the rendered slot.
- Images should be hosted on a CDN.
- Define the priority of the images and divide them into critical and non-critical assets.
- Lazy load below-the-fold images.
 - Use `<img loading="lazy">` for non-critical images.
- Load critical images early.
 - Give the LCP image high priority and preload it only when normal HTML discovery is too late; do not inline large product images into the HTML.
- **Adaptive loading** of images, loading high-quality images for devices on fast networks and using lower-quality images for devices on slow networks.

Exercise Beta

Which image policies improve product-page speed and visual stability? Select all that apply.

1

Prioritize the likely LCP product image while lazy-loading offscreen gallery media.

☐

2

Use the product name as the alternative text for every image in the gallery to keep descriptions consistent.

☐

3

Eagerly preload the full-resolution gallery after the hero request so later thumbnail selections feel instant.

☐

4

Declare dimensions or aspect ratios before image bytes arrive.

☐

5

Provide modern responsive sources sized for the rendered slot.

☐



Reveal answer

Check answer ↩

Form optimizations

Filling in forms is a huge part of the checkout flow, and a very troublesome one at that. It is the last step of the checkout process, and nailing a good checkout experience will greatly help the conversion rate.

Completing forms is especially painful on mobile devices, and extra attention has to be given to optimize forms for mobile. There are two kinds of forms to fill out during checkout: the shipping address form and the credit card form.

Country-specific address forms

Different countries have different address formats. To optimize for global shipping, having localized address forms helps greatly in improving conversions and ensures that users do not drop off when filling out the address forms because they do not understand certain fields. For example:

- "ZIP Code"s are called "Postal Code"s in the United Kingdom.
- There are no states in Japan, only prefectures.
- Different countries have their own postal/zip code formats and require different validation.

It is a hassle to have to find out this country-specific knowledge and build these forms yourself. This is where services like Stripe Checkout come in handy by providing a localized checkout form. Users will complete the rest of the payment flow on Stripe's platform.

Examples of Stripe checkout forms for different countries

US

UK

Shipping address

Name

United States

Address

Address line 2

City

ZIP

State

Shipping address

Name

United Kingdom

Address

Address line 2

Town or city

Postal code

Further reading: [Frank's Compulsive Guide to Postal Addresses](#) provides useful links and extensive guidance for address formats in over 200 countries.

Optimize autofilling

Filling up forms, especially long ones, is prone to typos. Most modern browsers have a feature called autofill, which helps users enter data faster and avoid filling up the same form data again by using values from similar forms filled previously.

Help users autofill their address forms by specifying the right `type` and `autocomplete` values for the form `<input>` s for the shipping address forms and credit card forms.

Shipping Address Form

Field	type	autocomplete	Others
Name	text	shipping name	autocorrect="off" spellcheck="false"
Country	Use <select>	shipping country	N/A
Address line 1	text	shipping address-line1	autocorrect="off" spellcheck="false"

Field	type	autocomplete	Others
Address line 2	text	shipping address-line2	autocorrect="off" spellcheck="false"
City	text	shipping address-level2	autocorrect="off" spellcheck="false"
State	Use <select>	shipping address-level1	N/A
Postal Code	text	shipping postal-code	autocorrect="off" spellcheck="false" and country-specific inputmode

Credit Card Form

Field	type	autocomplete	Others
Card Number	text	cc-number	autocorrect="off" spellcheck="false" inputmode="numeric"
Card Expiry	text	cc-exp	autocorrect="off" spellcheck="false" inputmode="numeric"
Card CVC	text	cc-csc	autocorrect="off" spellcheck="false" inputmode="numeric"

Notes

- Postal codes are not universally numeric. Use `inputmode="numeric"` only after the selected country establishes a digits-only format; otherwise keep a text keyboard and validate against that country's rules.
- `inputmode="numeric"` provides a hint to browsers for devices with onscreen keyboards to help them decide which keyboard to display. Unlike `<input type="number">`, `inputmode="numeric"` does not prevent users from typing non-

numeric values; it simply affects the keyboard being shown. For numeric card fields, `<input type="text" inputmode="numeric">` avoids quantity chevrons and still works with the `maxlength` / `minlength` / `pattern` attributes, which do not work with `<input type="number">`.

Read more about forms:

- [Autofilling | web.dev](#)
- [Autofill on Browsers: A Deep Dive | eBay Engineering Blog](#).

Alternative ways of address input

Instead of making users fill up a form containing granular address fields, there are other approaches that may be easier, at the cost of engineering complexity or reliance on external services:

1. **Address search/autocomplete:** Allow users to search for an address by typing in their street number and selecting from a list of suggestions. This reduces typos and is generally faster. However, users should still be given the option to override some values in case none of the suggestions are correct, which can happen due to an outdated address database. The Google Maps JavaScript API provides this via the [Place Autocomplete library](#).
2. **Selecting address location from a map:** Open up a map and allow users to pinpoint a location on the map. This is less common for checkout addresses and more common for ride-hailing applications.

Error messages

Leverage client-side validation and clearly communicate any form errors. Keep the associated live region empty initially; after validation fails, set `aria-invalid="true"` on the input and insert the message. Use a polite announcement by default and reserve assertive interruptions for urgent errors.

The example below shows the resulting state after validation has failed.

```
1 <form>
2   <div>
3     <label for="name">Name</label>
4     <input
```

```
5      required
6      type="text"
7      id="name"
8      name="name"
9      aria-invalid="true"
10     aria-describedby="name-error-message"
11   />
12   <span id="name-error-message" aria-live="polite" class="name-error-m
13     Enter the name to use for this order.
14   </span>
15 </div>
16 <button>Submit</button>
17 </form>
```

Source: [Help users find the error message for a form control](#) | [web.dev](#)

Focus states

Make the currently focused form control visually different from the other form inputs to help users identify which element is focused.

Best practices for payment and address forms

Read more about building good [Payment forms](#), [Address forms](#), and [general form best practices](#) on web.dev.

Exercise Beta

Which form behaviors reduce checkout friction without weakening correctness?
Select all that apply.

1

Validate fields without blocking typing and connect errors to the affected controls.

☐

2

Disable browser autofill so previously entered address data cannot overwrite the form's controlled state.

☐

3

Reset the form after a failed submission so the user cannot accidentally retry stale values.

☐

4

Retain safe form progress across navigation or recoverable submission failures.

☐

5

Use correct names, input types, and autocomplete tokens for address and payment fields.

☐

Reveal answer

Check answer ↩

Internationalization (i18n)

- Have pages translated into the supported languages.
 - Set the `lang` attribute on the `html` tag (e.g. `<html lang="zh-cn">`) to tell browsers and search engines the language of the page, which helps browsers offer a translation of the page.
 - Provide support for RTL languages by using CSS Logical Properties.
- Forms
 - Use a single form field for names.
 - Allow for various address formats. This is covered in the section above on address forms.

Read more:

- [Internationalization and localization | Forms | web.dev](#)
- [Internationalization | Design | web.dev](#)

Exercise Beta

Which responsibilities belong in an international commerce front end? Select all that apply.

1

Convert prices in the browser with a bundled exchange-rate table, then submit that amount at checkout.

☐

2

Concatenate translated fragments around localized dates, numbers, and plural words.

☐

3

Allow address fields, ordering, and validation to vary by country.

☐

4

Format server-provided amounts with the intended currency and locale.

☐

5

Support RTL and text expansion with logical properties and flexible controls.

☐



Reveal answer

Check answer ↩

Accessibility

- Use semantic elements where possible: headings, buttons, links, and inputs instead of styled `<div>` s.
- Give informative product images concise alternative text based on their purpose. Use `alt=""` only when an image is truly decorative or repeats adjacent information, not merely because a merchant omitted a description.
- Announce successful cart changes and inventory adjustments politely without moving focus.
- Building accessible forms has been covered in detail above. In summary:
 - `<input>` s should have associated `<label>` s.
 - `<input>` s are linked to their error messages via `aria-describedby` ; set `aria-invalid` after failure and announce new messages with an appropriate live region.
 - Use `<input>` s of the correct `type` s and appropriate validation-related attributes like `pattern` , `minlength` , `maxlength` .
 - Visual order matches DOM order.
 - Make the currently focused form control obvious.

Reference: [Accessibility | Forms | web.dev](#) and [WebAIM: Creating Accessible Forms](#)

Exercise Beta

Which practices make shopping and checkout accessible? Select all that apply.

1

Give quantity, variant, and form controls clear labels and programmatically associated errors.

☐

2

Handle product-card navigation on the card container while leaving the product title styled as non-interactive text.

☐

3

Structure navigation, main content, product sections, and checkout steps with semantic landmarks and headings.

☐

4

Use red and green outlines for stock and validation states while keeping the surrounding copy unchanged.

☐

5

Confirm cart changes without moving focus or interrupting unrelated work.

☐



Reveal answer

Check answer ↩

Security

Since payment details are highly sensitive, we have to make sure the website is secure:

- Use HTTPS so that all communication with the server is encrypted and so that other users on the same Wi-Fi network cannot intercept and obtain any sensitive details.
- The payment details submission API should not use HTTP `GET` because the sensitive details will be included as a query string in the request URL, which will get added to the browsing history, which is potentially unsafe if the browser is shared with other users. Use HTTP `POST` or `PUT` instead.
- Authorize every cart and order against the current user or guest-cart token, protect cookie-authenticated mutations from CSRF, escape untrusted text and sanitize only supported rich product or review markup, keep server-rendered output and third-party scripts under a strict CSP, and validate every product, price, inventory, address, and redirect value at its trust boundary.

! HTTPS-only, and keep payment fields out of URLs and analytics

Sensitive fields in query strings leak into browser history, referer headers, CDN access logs, and third-party analytics scripts, so GET is unsafe even over TLS. Serve

the whole site over HTTPS with HSTS, submit payment forms via POST, and scrub any inputs in checkout iframes from session-replay and error-tracking tools.

Source: [Security and privacy | web.dev](#)

Exercise Beta

Which responsibilities protect an e-commerce client and its mutation flows? Select all that apply.

- ☐ 1 Escape product and review content, sanitizing only explicitly supported rich markup.
- ☐ 2 Cache payment form values in local storage so a refresh can restore the checkout in progress.
- ☐ 3 Have the server revalidate identity, price, promotion, inventory, and order totals.
- ☐ 4 Sign the amount into a hidden field when rendering checkout and accept that value on submission to avoid another pricing lookup.
- ☐ 5 Collect card details in provider-hosted fields and send only an opaque payment-method ID to the application server.
- ☐ 6 Use appropriate CSRF and same-site protections for authenticated mutations.



Reveal answer

Check answer ↩

User experience

- Make the checkout page clean (e.g. minimal navbar and footer) and remove distractions to reduce bounce rate.
- Allow persisting cart contents (either in a database or cookies), as some people spend time researching and considering, only making the purchase during subsequent sessions. You don't want them to have to add all of the items to their cart again.

- Make promo code fields less prominent so that people without promo codes will not leave the page to search the web for a promo code. Those who have a promo code beforehand will take the effort to find the promo code input field.

✓ **Persist the cart server-side for signed-in users, cookie-backed for guests**

Carts are effectively purchase intent, and losing them across sessions or devices is a direct revenue hit. Tie the cart to the user ID for authenticated shoppers; for guests, persist the server cart and keep only its opaque handle in the secure cookie described above. If the interviewer focuses on identity continuity, discuss cart expiration and deterministic guest/account merging on sign-in.

Exercise Beta

Shoppers often return days later, and signed-in users may switch devices. How should cart continuity be persisted?

- 1 Put every cart line and calculated total into a long-lived cookie that accompanies all storefront requests.
- 2 Create a fresh cart whenever the shopper navigates between product, cart, and checkout pages.
- 3 Store the complete cart, prices, and inventory snapshot in local storage and restore it as authoritative.
- 4 Store the canonical cart on the server and resolve it by account or an opaque guest-cookie handle.



Reveal answer

Check answer ↩

Summary

E-commerce revenue comes from three levers a front-end design actually controls: how fast the first paint lands, how well catalog pages rank in organic search, and how little friction sits between an added item and a completed `POST /order`.

First paint? The PLP and PDP are server-rendered, so shoppers see product imagery, titles, and prices in the initial HTML response rather than after a hydration round trip. Critical above-the-fold assets are prioritized, responsive modern image formats are served through a CDN with below-the-fold lazy loading, and route-level code splitting keeps the JavaScript budget lean. Intent-based prefetching can reduce the perceived latency of the next click, and LCP, INP, and CLS are measured against the targets defined above.

Organic search? Catalog surfaces should be fast and crawlable, so SSR or pre-rendering is a strong default for `ProductList` and `Product` pages while Cart and Checkout can lean more on client rendering. Accurate `Product` JSON-LD makes eligible PDPs available for rich-result consideration, a generated `sitemap.xml` exposes canonical product pages, and route metadata keeps deep links and share previews coherent.

Checkout friction? The Client Store holds an optimistic `Cart` of `CartItem`s so the navbar badge and drawer respond instantly, but every mutation reconciles against the authoritative cart returned by the server. `price` and `total_price` remain server-owned, `GET /products` uses offset pagination, `AddressDetails` adapts to country-specific forms, and provider-hosted payment fields create an opaque `PaymentMethod` so raw card data never reaches `POST /order`.

Start with the conversion thesis, name SSR-for-catalog and CSR-for-transactional as the rendering split, and then spend the remaining time on server-owned pricing, optimistic-but-reconciled cart mutations, and the performance and form-UX work that closes the gap between a working prototype and a storefront people actually buy from.

References

The references below group e-commerce sources by the topic they inform, so it is easier to map each source back to the part of the article it relates to: real-world performance case studies, form and address input patterns, rendering strategy, and related system design articles.

Performance case studies

This subsection collects production postmortems from major e-commerce sites showing how page speed and Core Web Vitals improvements translated into concrete business metrics such as conversion and revenue.

- [Shopping for speed on eBay.com | web.dev](#) for eBay's end-to-end story of optimizing a large product catalog experience.
- [Speed by a thousand cuts | eBay Engineering](#) for the incremental wins that compound into a measurably faster shopping flow.
- [How Rakuten 24's investment in Core Web Vitals increased revenue per visitor by 53.37% | web.dev](#) for a concrete link between Core Web Vitals improvements and revenue per visitor.
- [How focusing on web performance improved Tokopedia's click-through rate by 35% | web.dev](#) for a large marketplace's mobile-first performance work.
- [Luxury retailer Farfetch sees higher conversion rates for better Core Web Vitals | web.dev](#) for how performance gains shift conversion at a high-end retailer.
- [How Renault improved its bounce and conversion rates | web.dev](#) for a long-funnel purchase flow where bounce and conversion were both sensitive to speed.
- [Lowe's website is among fastest performing e-commerce websites | web.dev](#) for the kinds of architecture choices that keep a catalog-heavy retailer fast.
- [JD.ID improves their mobile conversion rate by 53% with caching strategies | web.dev](#) for how caching and network strategy lift mobile conversion.
- [Milliseconds make millions | web.dev](#) for the broader research backing latency-to-revenue sensitivity across retailers.

Forms, autofill, and address input

This subsection covers the specific UX and browser integrations that make checkout forms fast and correct, especially around autofill and international addresses.

- [Autofill on browsers: a deep dive | eBay Engineering](#) for how browser autofill interacts with real-world checkout forms.
- [Autofilling | web.dev](#) for the standard guidance on wiring forms so browsers can autofill them reliably.
- [Address forms | web.dev](#) for conventions and pitfalls when designing address entry fields.

- [Payment and address form best practices | web.dev](#) for end-to-end guidance on checkout form quality.
- [Frank's compulsive guide to postal addresses](#) for background on the real-world variance in postal address formats across countries.

Rendering strategy

This subsection links the canonical overview that frames the rendering tradeoffs used throughout the architecture discussion.

- [Rendering on the web | web.dev](#) for the vocabulary and tradeoffs behind CSR, SSR, SSG, and hybrid approaches applied to e-commerce pages.

Related system design articles

The articles below overlap with this one on product surfaces, media-heavy pages, or input UX, and are useful cross-references when extending the e-commerce design.

- [Travel booking \(Airbnb\)](#) for another catalog-plus-detail-plus-checkout product with similar rendering and performance tradeoffs.
- [Photo sharing \(Instagram\)](#) for image-heavy rendering and media pipeline patterns that also apply to product imagery.
- [Autocomplete](#) for the interaction pattern powering product search suggestions on the storefront.

Exercises

Exercise 1 of 16 Beta

A storefront design focuses on the customer journey from discovery through purchase. Which boundaries keep that work focused? Select all that apply.

1

Define separate loading, responsiveness, layout-stability, and mutation-reliability targets for public catalog and checkout routes.

☐

2

Make seller catalog administration, fulfillment queues, and payout reporting part of the first-version storefront design.

☐

3

Treat personalized recommendation ranking and sponsored-product auctions as required parts of the baseline listing page.

☐

4

Support international shoppers across desktop and mobile, including checkout without mandatory account creation.

☐

5

Cover product listing and detail pages, a shared cart, checkout, and the successful order result.

☐

Reveal answer

Check answer ↩

Discussions



Normal T

B

I

U



<>

Aa



</>



Type comment here...

0/1000

Post



Report an issue



✓ Mark complete



Subscribe to our newsletter

Notify me

Practice

Get started

JavaScript functions

User interface coding

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Become an affiliate

Careers

Blog

Medium

DEV Community



English (US) ▾